THE DATA ENGINEER LAB

DATABRICKS FROM ZERO TO HERO

A 30-Day Interview & Career Readiness Series

DAY 27

MLflow — Databricks' ML Lifecycle Manager

SECTION 01

What MLflow Actually Is

MLflow is the open-source platform for managing the full machine learning lifecycle, covering experimentation, reproducibility, deployment, and a central model registry. On Databricks, it's deeply integrated rather than bolted on, backed by the same governance tools covered earlier in this series.

CORE CONCEPT

MLflow on Databricks spans three connected capabilities: Tracking, which logs every experiment's parameters and metrics; Model Registry, now backed by Unity Catalog, which centrally stores and versions trained models; and Model Serving, which deploys a registered model as a live REST endpoint.

SECTION 02

MLflow Tracking

Tracking is MLflow's experiment logging system. Every training run can automatically record its parameters, metrics, artifacts, and even the Databricks Runtime version it trained on, giving you a complete, comparable history across every experiment.

```
import mlflow

with mlflow.start_run():
    mlflow.log_param("n_estimators", 100)
    model = train_model(training_df)
    mlflow.log_metric("accuracy", 0.94)
    mlflow.sklearn.log_model(model, artifact_path="model")
```

SECTION 03

Model Registry — Now Backed by Unity Catalog

Once a model is trained and logged, it can be registered so other teams and systems can discover, version, and promote it.

IMPORTANT NUANCE

When models created with MLflow 3 are registered to the Unity Catalog model registry, you can view data such as parameters and metrics in one central location, across all experiments and workspaces. Registering to Unity Catalog instead of the legacy workspace model registry means the same catalog.schema.model namespace, access control, and lineage covered on Day 14 now extend to ML models as well.

```
import mlflow
```

```
mlflow.set_registry_uri("databricks-uc")

mlflow.register_model(
    "runs:/<run_id>/model",
    "catalog.schema.model_name"
)
```

- Each registered model can have multiple versions; the first registration is always version 1.
- Aliases let you refer to a specific version (such as 'champion' or 'production') via a stable URI, instead of hardcoding a version number.

SECTION 04

Model Serving — From Registry to Real-Time Predictions

A registered model in Unity Catalog isn't useful on its own until something can call it. Databricks Model Serving hosts machine learning models registered in the Unity Catalog model registry as REST endpoints, exposing them for real-time predictions without you needing to manage separate serving infrastructure.

```
# Download the registered model's artifacts, if needed locally
mlflow.artifacts.download_artifacts(
    f"models:/catalog.schema.model_name/1"
)

# The model is otherwise served directly as a REST endpoint
# via the Model Serving UI or API, no custom server required
```

SECTION 05

The Three Capabilities at a Glance

Capability	What It Does	Governed By
Tracking	Logs parameters, metrics, and artifacts per training run	Per-workspace experiment, viewable centrally with MLflow 3 + UC
Model Registry	Centrally stores, versions, and aliases trained models	Unity Catalog (catalog.schema.model)
Model Serving	Deploys a registered model as a live REST endpoint	Unity Catalog permissions on the underlying model

SECTION 06

The End-to-End ML Lifecycle on Databricks

```
DELTA LAKE TABLES  -->  FEATURE ENGINEERING
                                |
                                v
TRAINING NOTEBOOK  -->  MLFLOW TRACKING (params, metrics, artifacts)
                                |
                                v
            MODEL REGISTRY (Unity Catalog)
            catalog.schema.model, versioned, aliased
                                |
                                v
            MODEL SERVING ENDPOINT (REST API)
                                |
                                v
            REAL-TIME PREDICTIONS
```

REMEMBER THIS FOR INTERVIEWS

The strongest answer ties all three together: MLflow Tracking captures the experimentation history, the Unity Catalog model registry turns a chosen model into a governed, versioned asset, and Model Serving exposes that asset as a live endpoint, all sharing the same governance layer as the rest of the Lakehouse.

SECTION 07

Glossary — Day 27 Terms

Term	Definition
MLflow	The open-source platform for managing the full machine learning lifecycle: tracking, registry, and deployment.
MLflow Tracking	The component that logs parameters, metrics, and artifacts for each model training run.
Model Registry (Unity Catalog)	The centralized, governed store for registered, versioned ML models, addressed via catalog.schema.model.
Model Alias	A named pointer (such as 'champion') to a specific model version, used instead of hardcoding version numbers.
Model Serving	Databricks' capability to deploy a Unity Catalog-registered model as a live REST API endpoint.
Registered Model	A model that has been logged via Tracking and then registered to the Model Registry for versioning and governance.

SECTION 08

Interview Questions & Answers

Q1. What are the three core capabilities MLflow provides on Databricks?

A. Tracking, which logs experiment parameters and metrics; Model Registry, which centrally stores and versions trained models, now backed by Unity Catalog; and Model Serving, which deploys a registered model as a live REST endpoint.

Q2. How does registering a model to Unity Catalog differ from the legacy workspace model registry?

A. Registering to Unity Catalog places the model in the catalog.schema.model namespace, extending the same governance, access control, and lineage used for tables to ML models, and lets you view parameters and metrics in one central location across all experiments and workspaces.

Q3. What is a model alias, and why use one?

A. A named pointer, such as 'champion' or 'production', to a specific model version, used so downstream systems reference a stable alias rather than hardcoding a version number that changes every time the model is retrained.

Q4. What does mlflow.set_registry_uri('databricks-uc') actually do?

A. It points MLflow's model registry operations at the Databricks Unity Catalog model registry instead of a legacy workspace-scoped registry, so subsequent register_model calls store the model under Unity Catalog governance.

Q5. How does a registered model become available for real-time predictions?

A. Through Databricks Model Serving, which hosts machine learning models registered in the Unity Catalog model registry as REST endpoints, without requiring separate custom serving infrastructure.

Q6. Why is tying Model Registry to Unity Catalog significant from a governance standpoint?

A. Because it means ML models are no longer governed separately from the rest of the Lakehouse; the same catalog-level permissions, lineage, and audit capabilities that apply to tables and views now apply to models too.

STAY ON TRACK FOR ALL 30 DAYS

Comment the keyword **DATABRICKS** on the reel to get this document and future deep-dives sent directly to you.

Piyush Varma | The Data Engineer Lab